รายงานการทดลอง

Chapter 12: API Security & Chapter 13: OpenAPI with Swagger

VacQ Backend (Express + MongoDB)

ชื่อ-นามสกุล: _____ รหัสนิสิต: _____

1. Logout (ล้าง token cookie) [1 point]

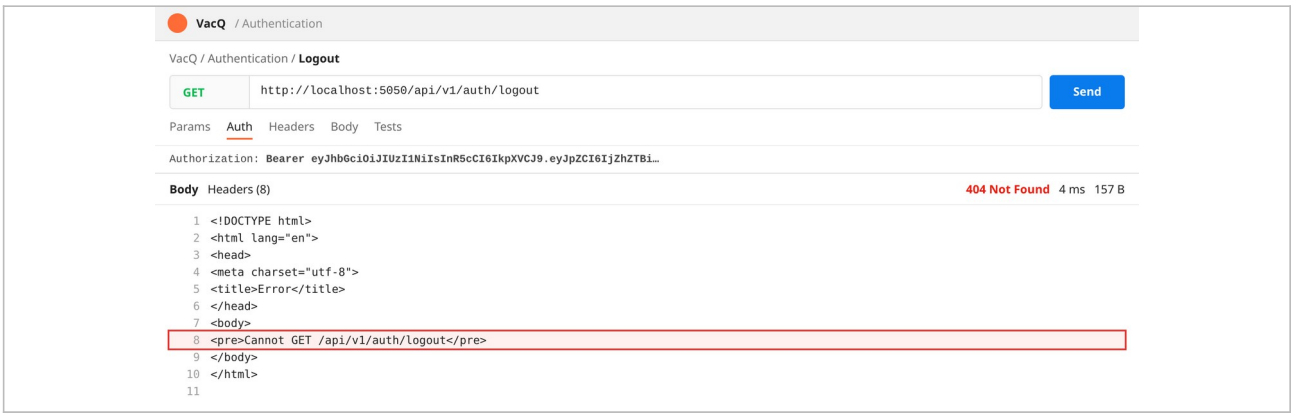
Code ที่เพิ่ม (controllers/auth.js, routes/auth.js, middleware/auth.js)

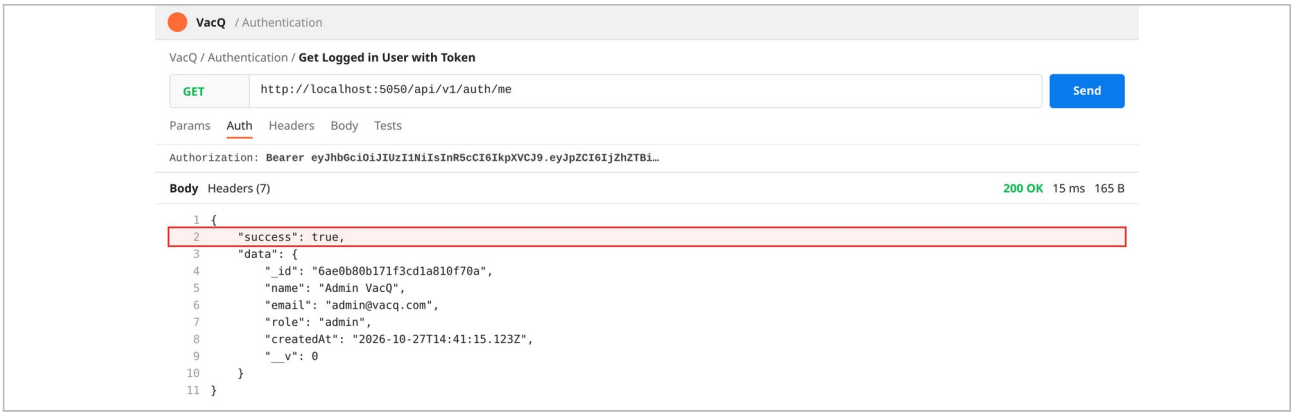
```
//controllers/auth.js  – try-catch ใน login (slide 3)
exports.login = async (req, res, next) => {
    try {
        const { email, password } = req.body;
        ...
        sendTokenResponse(user, 200, res);
    } catch (err) {
        return res.status(401).json({ success: false, msg: 'Cannot convert email or password to
string' });
    }
};

//@route GET /api/v1/auth/logout
exports.logout = async (req, res, next) => {
    res.cookie('token', 'none', { expires: new Date(Date.now() + 10 * 1000), httpOnly: true });
    res.status(200).json({ success: true, data: {} });
};

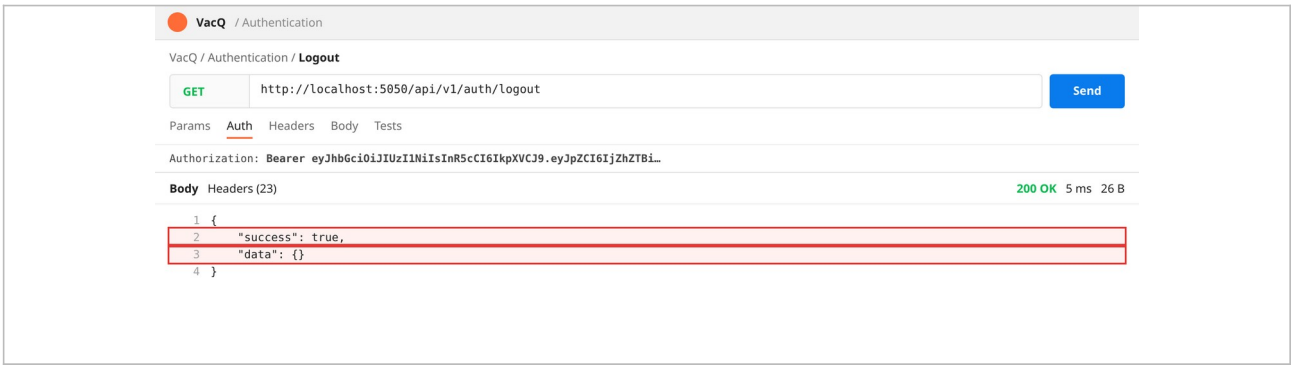
//middleware/auth.js
if (!token || token == 'null') { return res.status(401).json({ success: false, message: 'Not
authorize to access this route' }); }
```

BEFORE ยังไม่มี route logout → GET /auth/logout ได้ 404 "Cannot GET" และผู้ใช้ยังเรียก getMe ได้ (ยัง login อยู่)





AFTER กด Logout ได้ {"success": true, "data": {}} (Tests script: pm.environment.set("TOKEN", null))



AFTER จากนั้นกด Get Logged in User with Token อีกครั้ง → 401



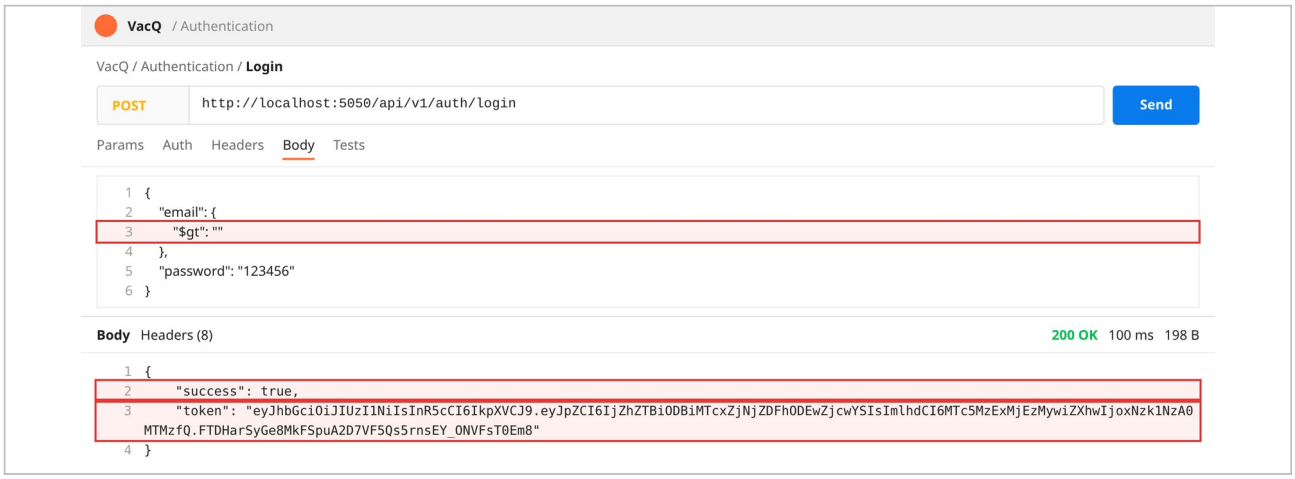
ผลลัพธ์: หลัง logout ไม่สามารถเข้าถึง route ที่ต้อง login ได้ ได้ "message": "Not authorize to access this route" = Logout สำเร็จ

2. Prevent NoSQL Injection & Sanitize Data [1 point]

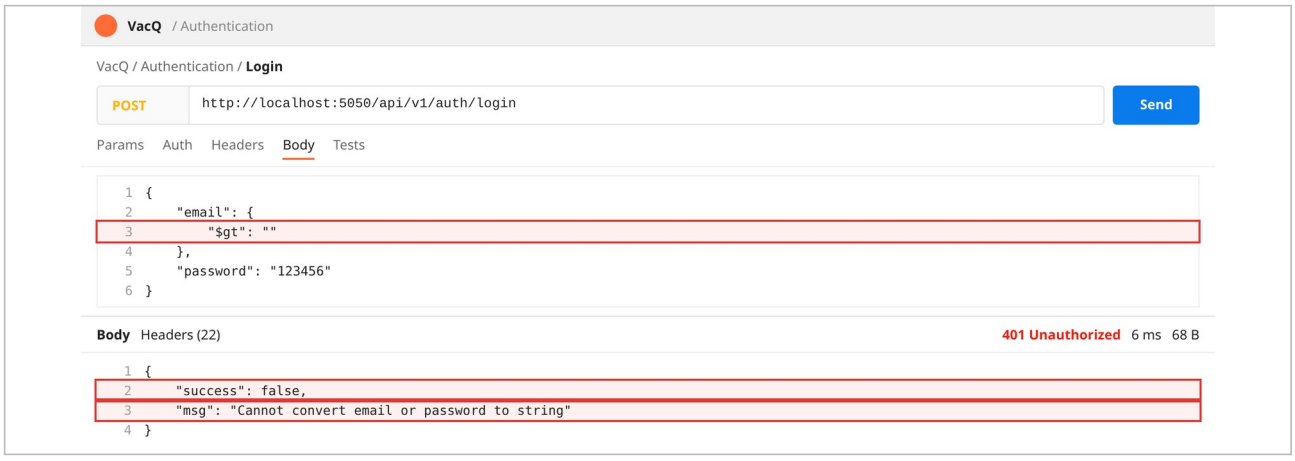
Code ที่เพิ่ม

```
//server.js
const mongoSanitize = require('express-mongo-sanitize');
app.use(express.json());
app.use(cookieParser());
//Sanitize data
app.use(mongoSanitize());
```

BEFORE Login ด้วย "email": {"$gt": ""} → findOne ดึง user คนแรก (admin) และ login สำเร็จ ได้ token



AFTER Request เดิม → $gt ถูกลบออก ถูก catch ไว้ได้ 401 "Cannot convert email or password to string"



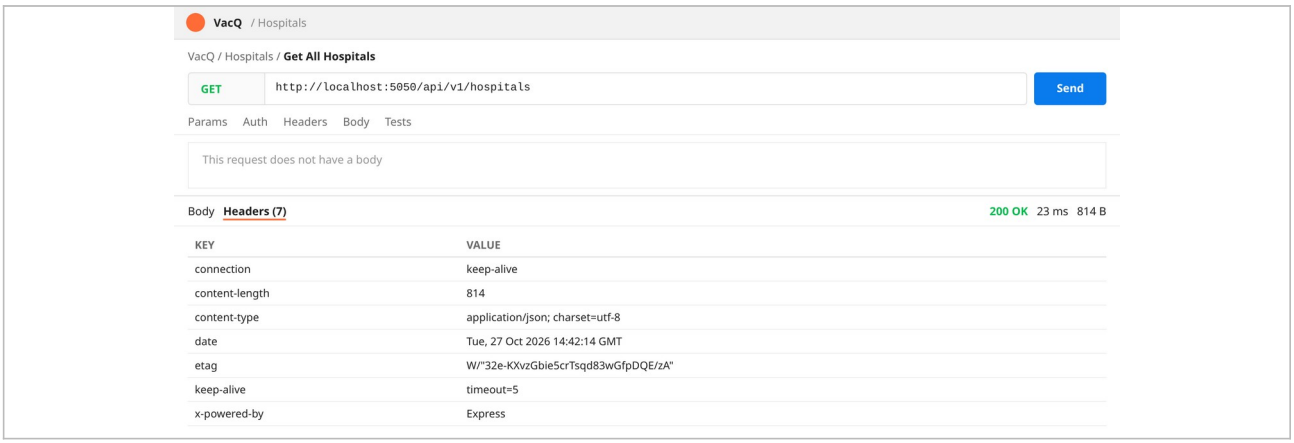
ผลลัพธ์: ป้องกัน NoSQL injection สำเร็จ ไม่ได้ token กลับมา

3. Security Headers (Helmet) [1 point]

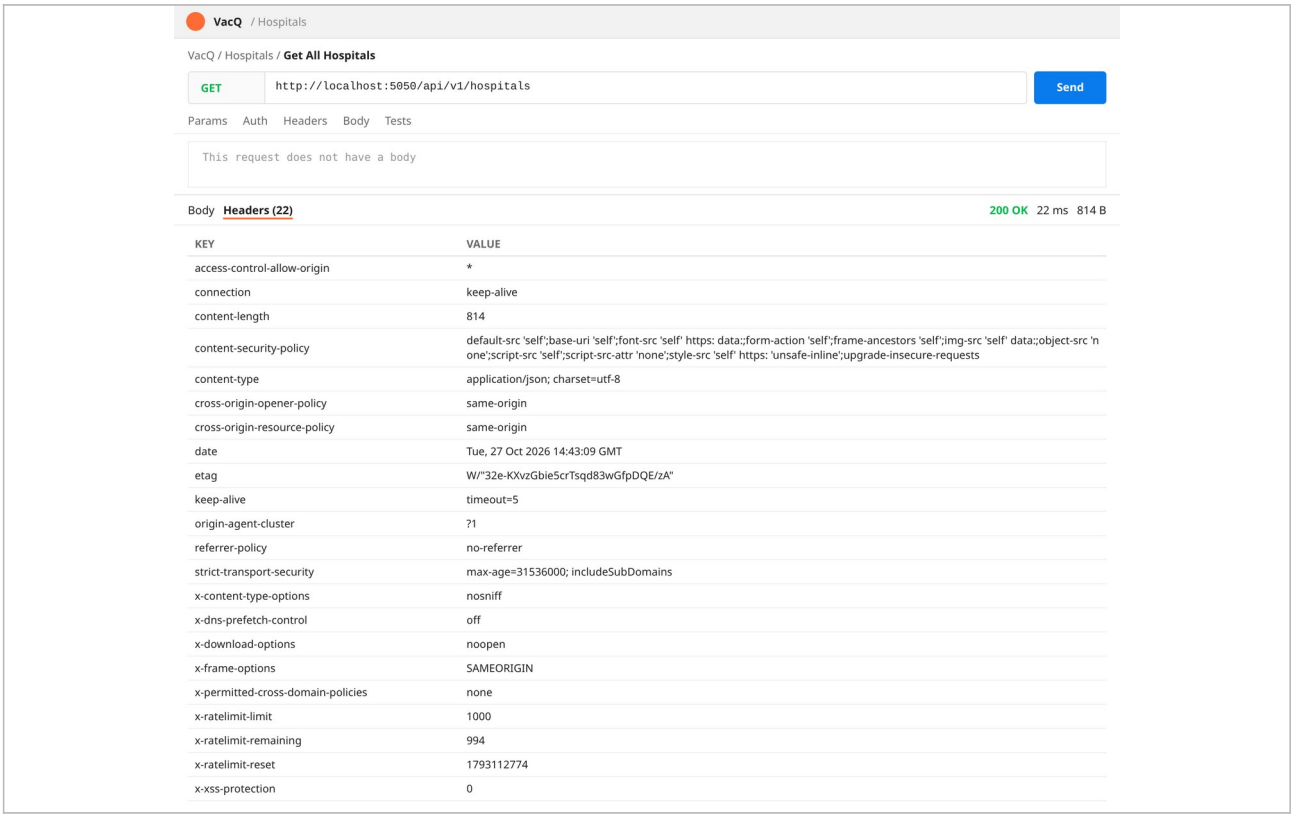
Code ที่เพิ่ม

```
//server.js
const helmet = require('helmet');
//Set security headers
app.use(helmet());
```

BEFORE GET /hospitals → Response headers มีแค่ 7 รายการ และเปิดเผย X-Powered-By: Express



AFTER GET /hospitals → มี Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options ฯลฯ และไม่มี X-Powered-By



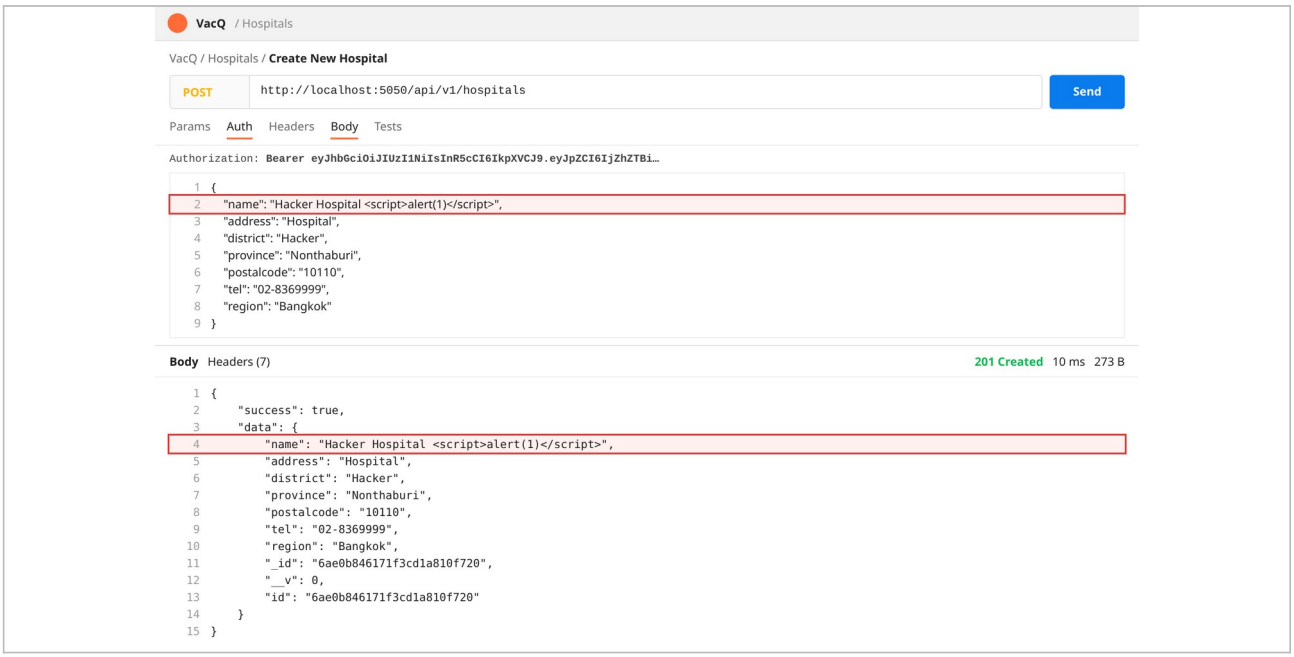
ผลลัพธ์: Helmet เพิ่ม security headers ให้ทุก response สำเร็จ

4. XSS Protection [1 point]

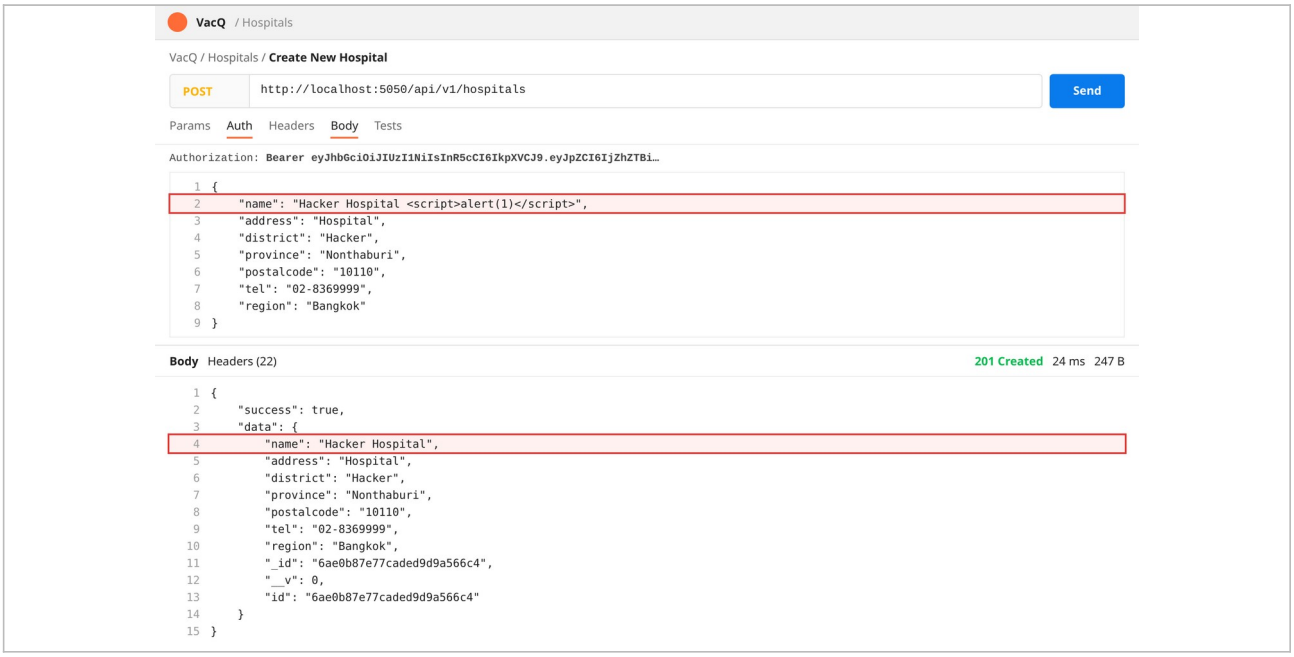
Code ที่เพิ่ม

```
//server.js
const { xss } = require('express-xss-sanitizer');
//Prevent XSS attacks
app.use(xss());
```

BEFORE สร้าง Hospital ชื่อ "Hacker Hospital <script>alert(1)</script>" → script ถูกบันทึกลง DB ทั้งหมด



AFTER Request เดิม (ลบ รพ. เดิมออกก่อน) → ส่วน <script>...</script> ถูกตัดออก เหลือ "Hacker Hospital"



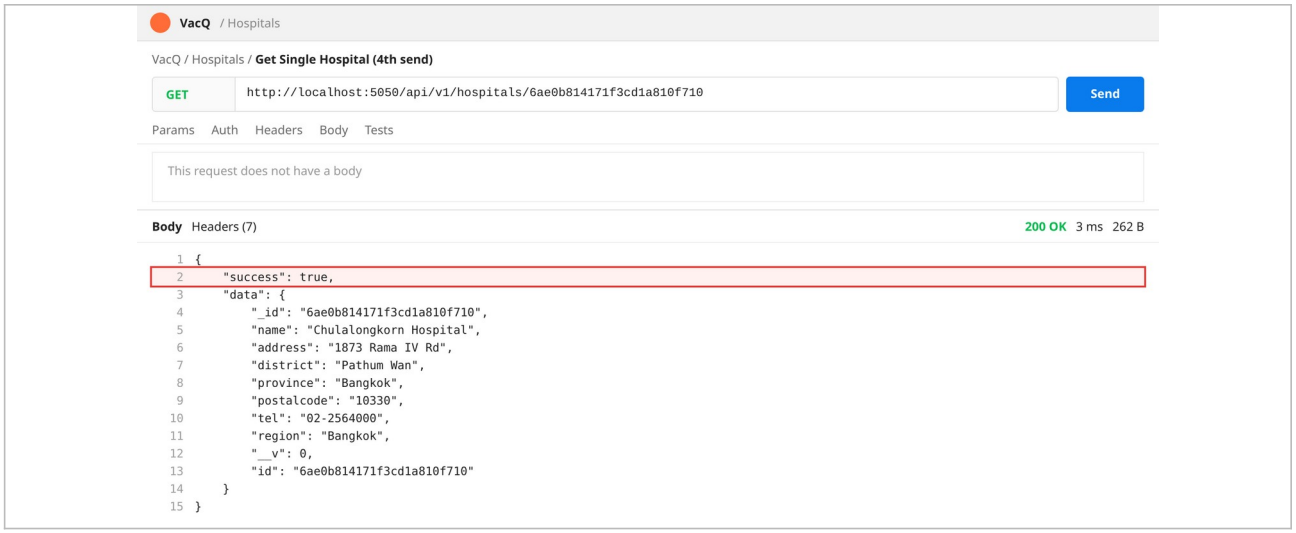
ผลลัพธ์: ป้องกัน XSS สำเร็จ

5. Rate Limiting (express-rate-limit) [1 point]

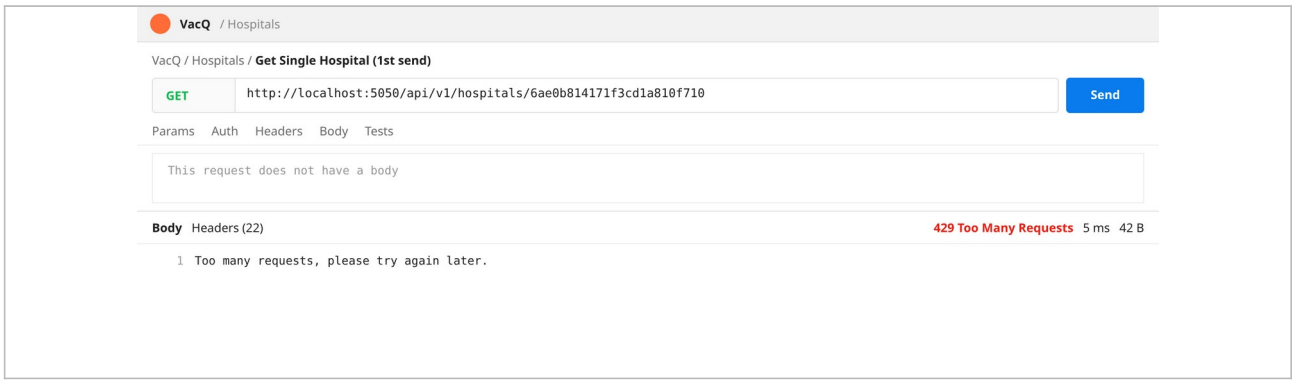
Code ที่เพิ่ม

```
//server.js
const rateLimit = require('express-rate-limit');
//Rate Limiting
const limiter = rateLimit({
    windowMs: 10 * 60 * 1000, //10 mins
    max: 1   // ตั้งเป็น 1 เพื่อทดสอบ แล้วเปลี่ยนกลับเป็น 100
});
app.use(limiter);
```

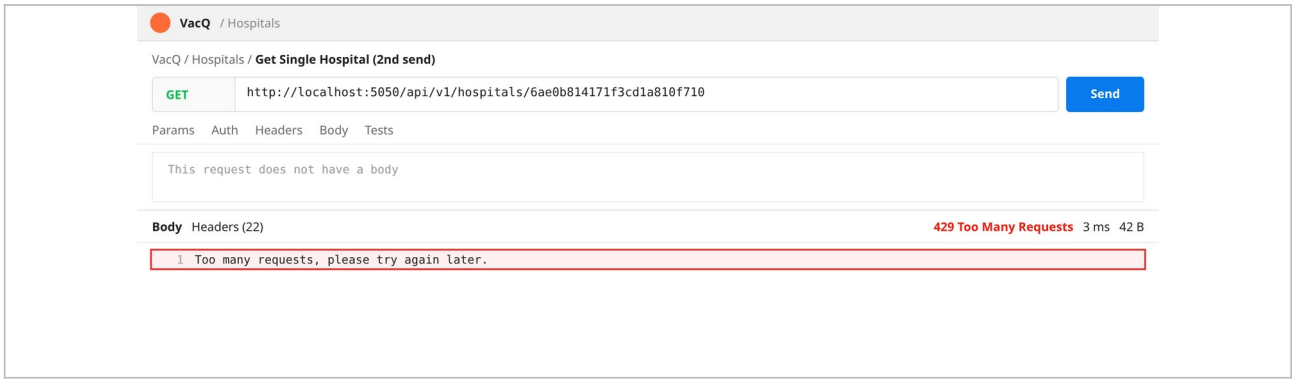
BEFORE ยังไม่มี rate limit → กด Send Get Single Hospital ซ้ำ (ครั้งที่ 4) ก็ยังได้ 200 OK



AFTER max = 1 (ภายใน 10 นาที): ครั้งแรกได้ 200 OK



AFTER ครั้งที่ 2 → 429 Too Many Requests



ผลลัพธ์: จำกัดจำนวน request สำเร็จ (หลังทดสอบเปลี่ยน max กลับเป็น 100)

6. OpenAPI Document: Hospital tag + 5 operations [3 points]

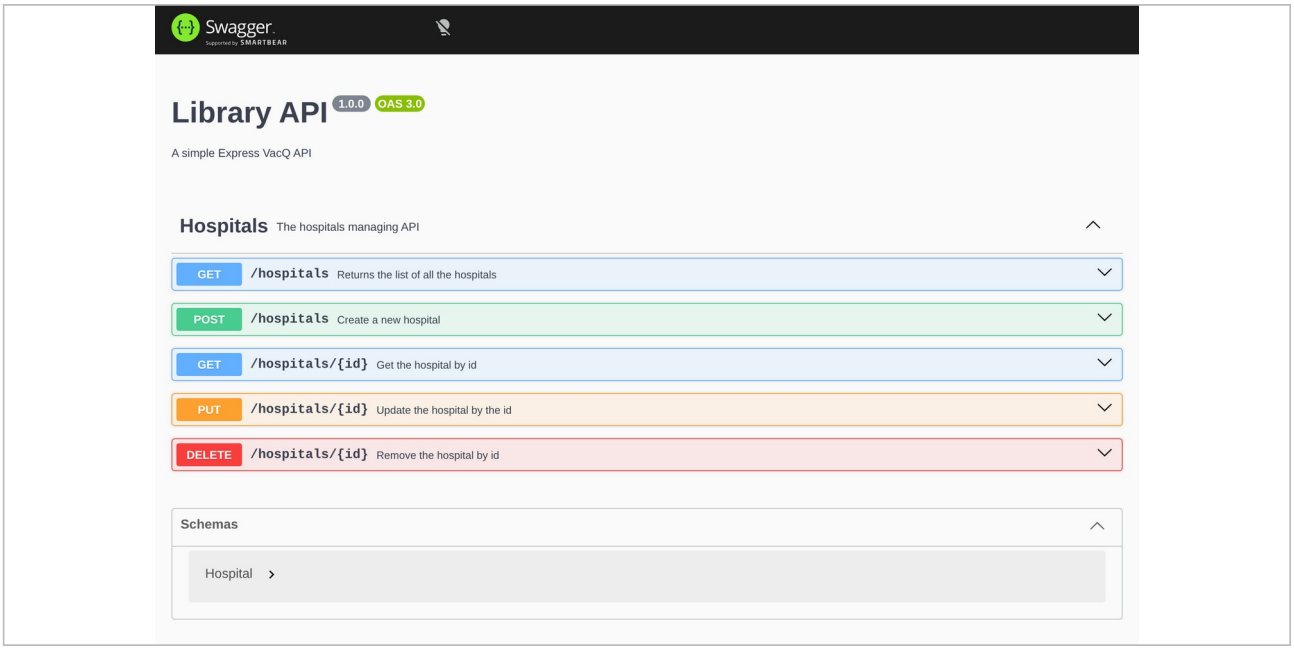
routes/hospitals.js (ย่อ — ดู code ฉบับเต็มใน repository)

```
/**
 * @swagger
 * tags:
 *   name: Hospitals
 *   description: The hospitals managing API
 */
/**  @swagger  /hospitals:          get    – Returns the list of all the hospitals   */
/**  @swagger  /hospitals/{id}:     get    – Get the hospital by id                  */
/**  @swagger  /hospitals:          post   – Create a new hospital                   */
/**  @swagger  /hospitals/{id}:     put    – Update the hospital by the id           */
/**  @swagger  /hospitals/{id}:     delete – Remove the hospital by id               */
/**  + components/schemas/Hospital                                                    */
```

BEFORE http://localhost:5050/api-docs/ ก่อนเพิ่ม swagger comment → "No operations defined in spec!"



AFTER หลังเพิ่ม → มี tag Hospitals และ GET /hospitals, POST /hospitals, GET /hospitals/{id}, PUT /hospitals/{id}, DELETE /hospitals/{id} พร้อม Schema Hospital



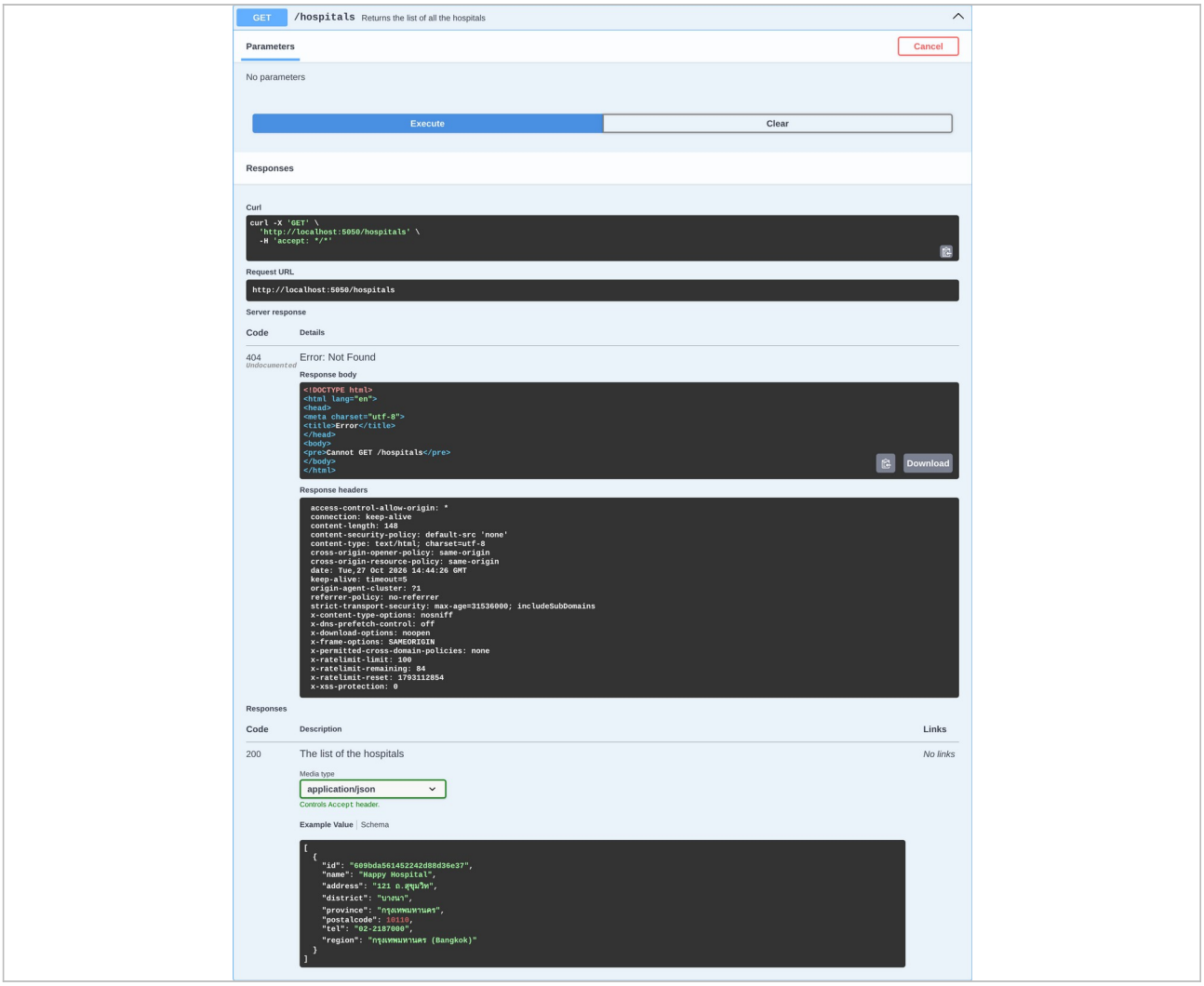
ผลลัพธ์: เพิ่ม Hospital tag, get all, get one, add, update และ delete hospital ลงใน OpenAPI document สำเร็จ

7. Server URL + Execute get-all-hospitals ผ่าน Api-Docs [2 points]

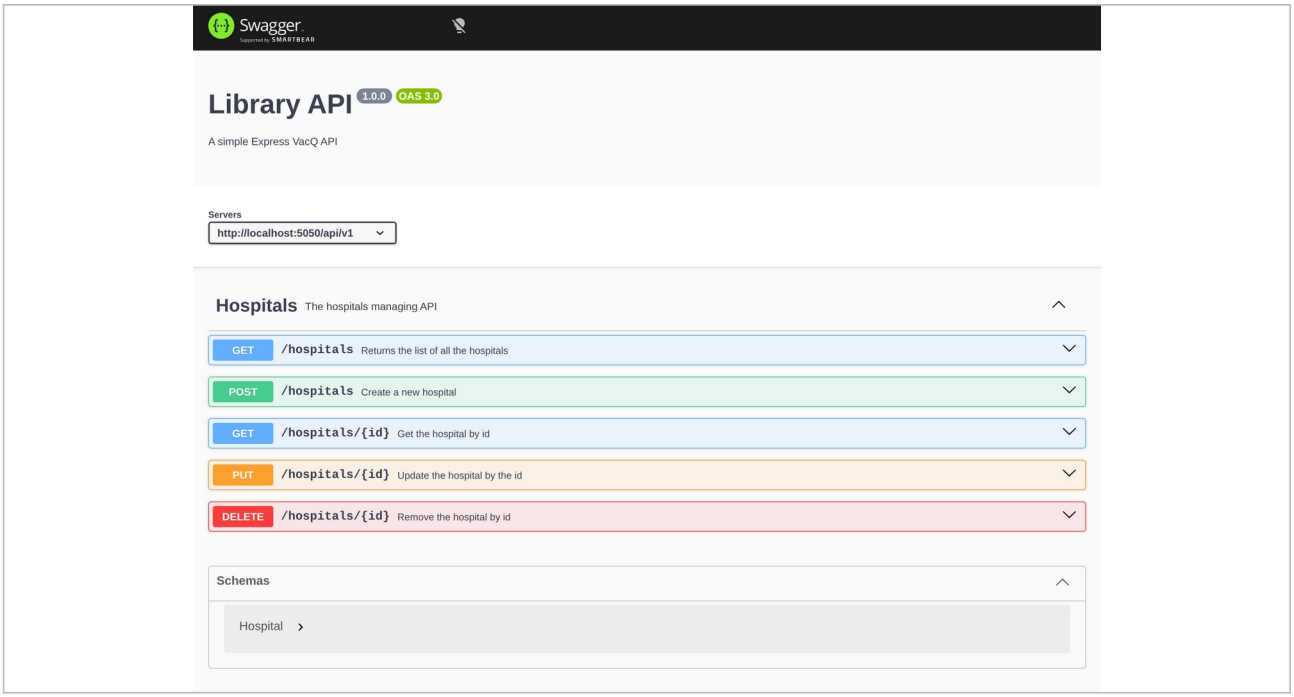
Code ที่เพิ่ม

```
//server.js
const swaggerOptions = {
    swaggerDefinition: {
        openapi: '3.0.0',
        info: { title: 'Library API', version: '1.0.0', description: 'A simple Express VacQ API' },
        servers: [
            { url: 'http://localhost:5050/api/v1' }
        ],
    },
    apis: ['./routes/*.js'],
};
```

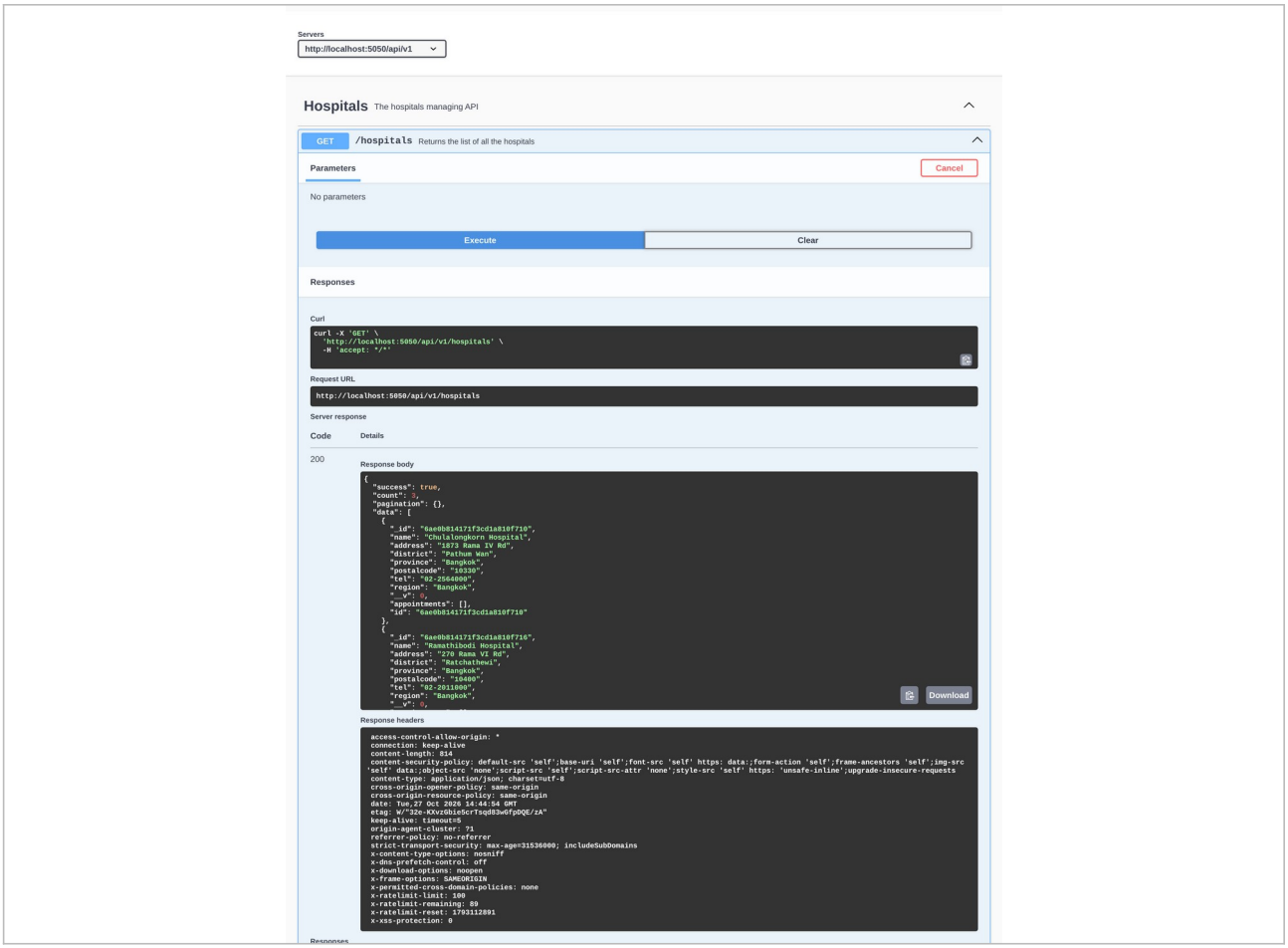
BEFORE ยังไม่มี servers → ไม่มี dropdown Servers, Try it out → Execute ยิงไปที่ http://localhost:5050/hospitals ได้ 404 Not Found



AFTER มี Servers: http://localhost:5050/api/v1



AFTER GET /hospitals → Try it out → Execute → Request URL http://localhost:5050/api/v1/hospitals ได้ 200 พร้อมรายการโรงพยาบาล



ผลลัพธ์: เพิ่ม server url และ execute get-all-hospitals ผ่าน Api-Docs สำเร็จ